國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 07

姓名： 紀宥丞

學號： 114310221

Lab07-Ex1.
Convert Infix Expression to Postfix Using a Stack Implemented with Linked List
Code

```
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;

// 定義 Stack 的節點結構
struct Node {
    char data;   // 存儲字符
    Node* next; // 指向下一個節點
};

// 使用 linked list 實作 Stack
class Stack {
private:
    Node* top; // 指向堆疊頂端
public:
    Stack() { top = nullptr; } // 初始化堆疊

    // Push
    void push(char ch) {
        Node* newNode = new Node; // 創建新節點
        newNode->data = ch;          // 存入數據
        newNode->next = top;         // 讓新節點指向目前的頂端
        top = newNode;               // 更新頂端指標
    }

    // Pop
    char pop() {
        if (isEmpty()) return '\0'; // 若堆疊為空，回傳空字符
        char ch = top->data;         // 取得頂端元素
        Node* temp = top;            // 暫存頂端節點
        top = top->next;             // 移動頂端指標
        delete temp;                 // 釋放記憶體
```

```
        return ch;                          // 回傳彈出的元素
    }

    // Peek
    char peek() {
        return (top ? top->data : '\0');
    }

    // 判斷堆疊是否為空
    bool isEmpty() {
        return top == nullptr;
    }
};

// 判斷運算子(加減乘除) 的優先順序
int precedence(char op) {
    if (op == '*' || op == '/') {
        return 2; // 乘除優先權較高
    } else if (op == '+' || op == '-') {
        return 1; // 加減優先權較低
    }
    return 0; // 其他字元 (如左括號) 回傳 0
}

// 將中序表達式 (infix) 轉換為後序表達式 (postfix)
void InfixToPostfix(const char* infix, char* postfix) {
    Stack s;
    int i = 0, j = 0;

    // 1. 由左至右讀進中序運算式的每個字元
    while (infix[i] != '\0') {
        char token = infix[i];

        // 2. 如果讀進的字元為運算元，則直接輸出到後序式中
        if (isalnum(token)) {
            postfix[j++] = token;
        }
        // 遇到左括號 '('，因為在堆疊外優先權最高，直接 push 進入堆疊
```

```
        else if (token == '(') {
            s.push(token);
        }
        // 3. 如果遇到右括號 ')'，則 pop 堆疊內的運算子，直到遇到左括號 '('
        else if (token == ')') {
            while (!s.isEmpty() && s.peek() != '(') {
                postfix[j++] = s.pop();
            }
            if (!s.isEmpty() && s.peek() == '(') {
                s.pop(); // 將左括號彈出，但不輸出到後序式
            }
        }
        // 5. 遇到運算子 (+, -, *, /) 時的處理
        else {
            // 與堆疊頂端的運算子比較優先權：如果外面的優先權較小或等於時，則 pop 並輸出
            while (!s.isEmpty() && precedence(s.peek()) >= precedence(token)) {
                postfix[j++] = s.pop();
            }
            // 再把外面這個運算子 push 進去堆疊
            s.push(token);
        }
        i++;
    }

    // 6. 中序式讀完後，如果運算子堆疊不為空，則將其內的運算子逐一 pop 出來，輸出到後序式
    while (!s.isEmpty()) {
        postfix[j++] = s.pop();
    }

    // 加上字串結束字元
    postfix[j] = '\0';
}

int main() {
    char infix[100], postfix[100];
    cout << "Enter an Infix expression: ";
```

```
    cin >> infix; // 輸入中序表達式

    InfixToPostfix(infix, postfix); // 轉換為後序表達式
    cout << "Postfix expression: " << postfix << endl; // 輸出後序表達式

    return 0;
}
```

Discussion Section

(A+B)*D+E/(F+A*D)+C							
Token	Stack [0]	[1]	[2]	[3]	[4]	Top	Output
(	(					0	
A	(					0	A
+	(	+				1	A
B	(	+				1	AB
)						-1	AB+
*	*					0	AB+
D	*					0	AB+D
+	+					0	AB+D*
E	+					0	AB+D*E
/	+	/				1	AB+D*E
(	+	/	(			2	AB+D*E
F	+	/	(			2	AB+D*EF
+	+	/	(	+		3	AB+D*EF
A	+	/	(	+		3	AB+D*EFA
*	+	/	(	+	*	4	AB+D*EFA
D	+	/	(	+	*	4	AB+D*EFAD
)	+	/				1	AB+D*EFAD*+
+	+					0	AB+D*EFAD*+/+
C	+					0	AB+D*EFAD*+/+C
<eos>						-1	AB+D*EFAD*+/+C+

A+B*(C*(D+E))								
Token	Stack [0]	[1]	[2]	[3]	[4]	[5]	Top	Output
A							-1	A
+	+						0	A
B	+						0	AB
*	+	*					1	AB
(	+	*	(				2	AB
C	+	*	(				2	ABC
*	+	*	(	*			3	ABC
(	+	*	(	*	(		4	ABC
D	+	*	(	*	(		4	ABCD
+	+	*	(	*	(	+	5	ABCD
E	+	*	(	*	(	+	5	ABCDE
)	+	*	(	*			3	ABCDE+
)	+	*					1	ABCDE+*
<eos>							-1	ABCDE+**+

3+1*2+(4+5*6)/(9-7)						
Token	Stack [0]	[1]	[2]	[3]	Top	Output
3					-1	3
+	+				0	3
1	+				0	31
*	+	*			1	31
2	+	*			1	312
+	+				0	312*+
(	+	(			1	312*+
4	+	(			1	312*+4
+	+	(	+		2	312*+4
5	+	(	+		2	312*+45
*	+	(	+	*	3	312*+45
6	+	(	+	*	3	312*+456
)	+				0	312*+456*+
/	+	/			1	312*+456*+
(	+	/	(		2	312*+456*+
9	+	/	(		2	312*+456*+9
-	+	/	(	-	3	312*+456*+9
7	+	/	(	-	3	312*+456*+97
)	+	/			1	312*+456*+97-
<eos>					-1	312*+456*+97-/+